



Proyecto Final del modulo 5.

Raúl Llamosas Alvarado

Eumir Rendon Uresti
Randy Jesús García Mejía

Gabriela Peña Franco

Daniela Ibarra Gatica
Daniela Vázquez Sánchez

Table of contents

Introducción	1
Problema a resolver	1
Metodología	1
EDA	2
Codificación de variables categóricas.	4
Escalamiento de variables.	5
Modelado	5
Regresión Logística.	5
Modelos de Ensamble y Redes.	8
Interpretación de resultados y conclusión	15
Conclusión	16

Todo nuestro código se encuentra en el [siguiente repositorio de GitHub](#), al cual le hemos dado acceso a los siguiente correos:

- [Alan Riva Palacio Cohen](#).
- [Jorge Ignacio Gonzalez Cazares](#)

El proyecto está ordenado de la siguiente forma:

```
+-- __init__.py
+-- README.md
+-- requirements.txt
+-- src
|   +-- EDA
|   |   +-- clean_dataset.py
|   |   +-- EDA.ipynb
|   |   +-- __init__.py
|   |   +-- seoul_bike_eda_report.html
|   +-- entregables
|   |   +-- presentacion.qmd
|   |   +-- reporte.html
|   |   +-- reporte.pdf
|   |   +-- reporte.qmd
|   +-- estructura.py
+-- ML
|   +-- __init__.py
|   +-- modelaje.py
|   +-- showcase_of_results.ipynb
```

Separamos la limpieza del dataset y el modelaje en dos .py usando programación orientada a objetos para que después sean llamados dentro de los .ipynb usando sus respectivos métodos de clase.

Introducción

Problema a resolver

Pronóstico de Demanda Espaciotemporal para Bicicletas Compartidas

- Idea Principal: Predecir la cantidad de bicicletas alquiladas por hora en función de variables meteorológicas e indicadores temporales.
- Base de Datos: Seoul Bike Sharing Demand (UCI).

Metodología

- Limpieza + EDA: codificación de variables, creación de variables categóricas y creación de variables cíclicas.
- Escalamiento usando **Standard Scaler**
- Modelación usando distintos modelos
- Métricas con MAE, MSE, RMSE, R^2 , R^2 -ajustado.

EDA

Vamos a importar el dataset de ‘Seoul Bike Sharing Demand’ del repositorio de Machine Learning de UC Irvine, que [podemos encontrar aquí](#) siguiendo la documentación de importación que en dicha URL se encuentra.

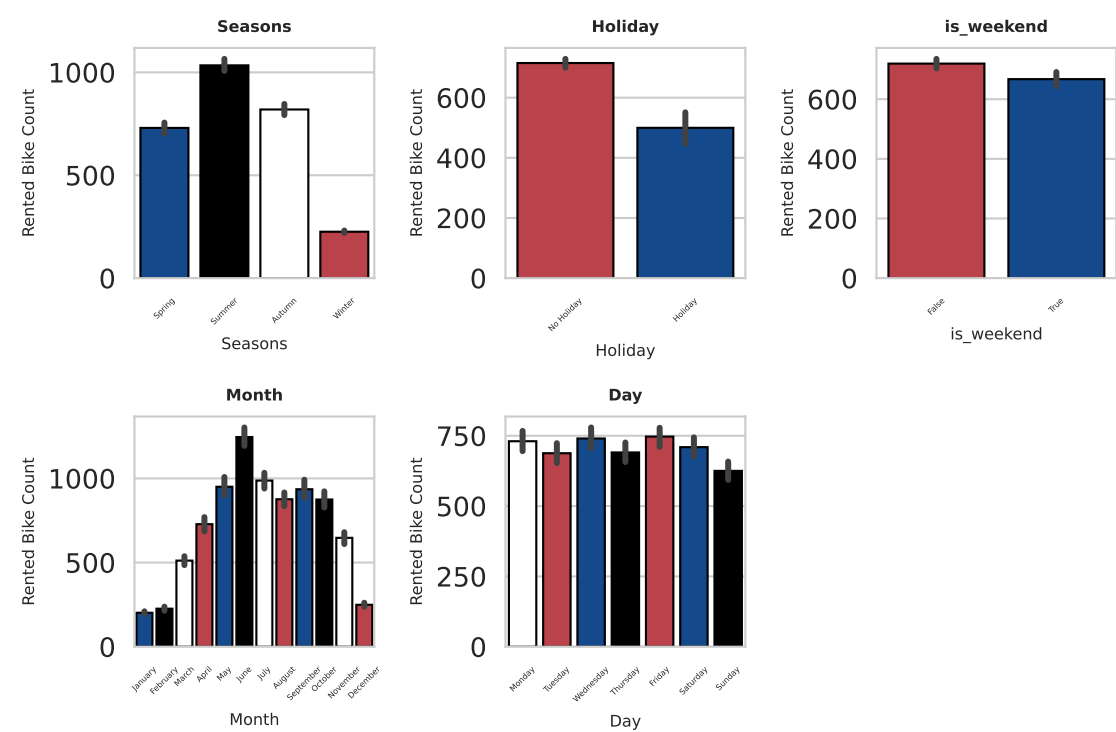
Vamos a limpiar el dataset usando `clean_dataset.py`

Y después haremos un EDA inicial con `Profile Report`.

Dataset statistics	
Number of variables	16
Number of observations	8760
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	1.0 MiB
Average record size in memory	121.0 B

Figure 1: profile report

Podemos notar que no tiene NAs en las filas. Además del EDA inicial queremos ilustrar las siguientes observaciones:



Con base en lo anterior, las observaciones son:

- La renta de bicicletas aumenta en verano y otoño (debido a que son condiciones más agradables para rodar)
- Se rentan más bicicletas en días que no son festivos (la gente las usa para ir a trabajar, probablemente)
- Se rentan más o menos en una misma cantidad entre fin de semana o entre semana.
- La gráfica de los meses nos muestra la tendencia de las temporadas.
- El día con menos rentas es domingo (probablemente porque la gente descansa ese día)

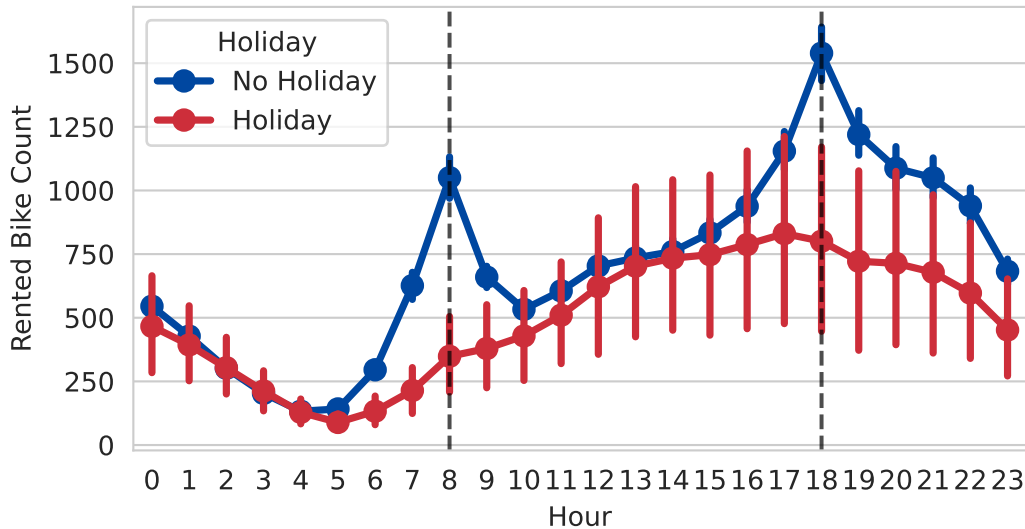
```
plt.figure(figsize=(6,3))
sns.pointplot(x=seoul_bike_sharing_demand_df['Hour'],
              y=seoul_bike_sharing_demand_df['Rented Bike Count'],
              hue=seoul_bike_sharing_demand_df['Holiday'],
              palette={"Holiday": "#CD2E3A", "No Holiday": "#0047A0"})
```

```
plt.title("Patrón de rentas de bicicletas basado en hora discriminando por día festivo.")

# Add vertical lines at hour 8 and hour 18
plt.axvline(x=8, color='black', linestyle='--', linewidth=1.5, alpha=0.7)
plt.axvline(x=18, color='black', linestyle='--', linewidth=1.5, alpha=0.7)

plt.show()
```

Patrón de rentas de bicicletas basado en hora discriminando por día festivo.

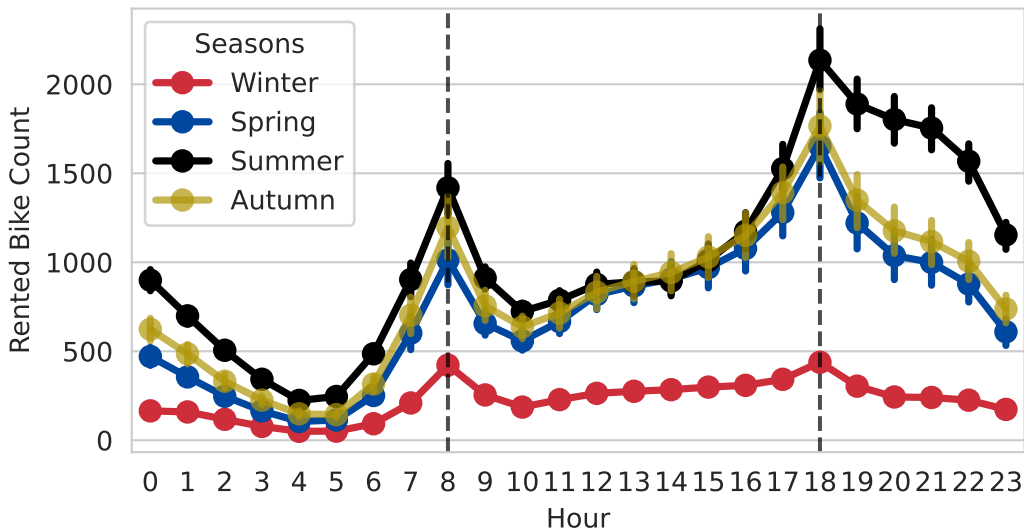


Observaciones de la gráfica de arriba serían:

- En días no festivos los picos se encuentran en las horas de entrada y salida de las oficinas.
- En días festivos no hay tales picos es una gráfica más suave que va disminuyendo conforme acaba el día.

Nótese que las líneas de variabilidad son mucho más grandes en días festivos mientras que en no festivos son mucho más cortas, lo que puede respaldar que el servicio es usado de manera regular por trabajadores para trasladarse hacia o desde la oficina

Patrón de rentas por hora tomando en cuenta temporadas



Observaciones:

- Los picos del horario de oficina permanecen independientes de la estación, no obstante, en invierno la gente renta menos bicicletas debido a que son condiciones más adversas para rodar al presentar nevadas o ventiscas durante la estación.
- En verano y otoño se rentan más.

```
df1 = seoul_bike_sharing_demand_df["Dew point temperature"]
df2 = seoul_bike_sharing_demand_df["Temperature"]

side_by_side = pd.concat([df1, df2], axis=1)
side_by_side
```

	Dew point temperature	Temperature
0	-17.6	-5.2
1	-17.6	-5.5
2	-17.7	-6.0
3	-17.6	-6.2
4	-18.6	-6.0
...	...	...
8755	-10.3	4.2
8756	-9.9	3.4
8757	-9.9	2.6
8758	-9.8	2.1
8759	-9.3	1.9

Las dos columnas de arriba están muy correlacionadas entre sí y no aportan información muy distinta. Las dos son de temperatura. Eliminaré la primera, 'Dew Point Temperature'

Codificación de variables categóricas.

Después de haber hecho el EDA procedemos a codificar las variables categóricas para que estén listas para el modelado. Lo hacemos de la siguiente manera

```
def replace_holiday(self):
    self.dataframe['Holiday'] = self.dataframe['Holiday'].apply(
        lambda x: 1 if x=="Holiday"
        else 0 )

def replace_functioning_day(self):
    self.dataframe['Functioning Day'] = self.dataframe['Functioning Day'].apply(
        lambda x: 1 if x=="Yes"
        else 0)

def encode(self):
    season_map = {'Spring': 0, 'Summer': 1, 'Autumn': 2, 'Winter': 3}
    self.dataframe['Seasons'] = self.dataframe['Seasons'].map(season_map)
    day_map = {'Monday': 0, 'Tuesday': 1, 'Wednesday': 2,
               'Thursday': 3, 'Friday': 4, 'Saturday': 5, 'Sunday': 6}

    month_map = {'January': 0, 'February': 1, 'March': 2, 'April': 3,
                 'May': 4, 'June': 5, 'July': 6, 'August': 7,
                 'September': 8, 'October': 9, 'November': 10, 'December': 11}
    functioning_day_map = {"Yes":1,"No":0}

    self.dataframe['Day'] = self.dataframe['Day'].map(day_map)
    self.dataframe['Month'] = self.dataframe['Month'].map(month_map)
    self.dataframe["Functioning Day"]=self.dataframe["Functioning Day"].map(functioning_day_map)
```

También, tomamos en cuenta la estacionalidad temporal al usar la siguiente función

```
def make_sin_cos(self):
    self.dataframe['hour_sin'] = np.sin(2 * np.pi * self.dataframe['Hour'] / 24)
    self.dataframe['hour_cos'] = np.cos(2 * np.pi * self.dataframe['Hour'] / 24)
    self.dataframe['dow_sin'] = np.sin(2 * np.pi * self.dataframe['Day'] / 7)
    self.dataframe['dow_cos'] = np.cos(2 * np.pi * self.dataframe['Day'] / 7)

    self.dataframe['month_sin'] = np.sin(2 * np.pi * self.dataframe['Month'] / 12)
    self.dataframe['month_cos'] = np.cos(2 * np.pi * self.dataframe['Month'] / 12)
```

De esta manera, las 23 horas están “cerca” de las 1:00AM del siguiente día. Tomando así en cuenta la estacionalidad temporal de nuestro dataset. Que resultará ser muy importante después.

Todas estas modificaciones de arriba más la descarga del dataset las reunimos en la siguiente función para poder ser llamada en otros archivos, particularmente en la clase que hace todo el modelado. Se llama de la siguiente manera:

```
def run_clean(self):
    self.make_date()
    self.make_weekend()
    self.drop_unnecesary_columns()
    self.replace_holiday()
    self.encode()
    self.make_sin_cos()
    output_df = self.dataframe
    return output_df
```

Un ejemplo de como se manda a llamar es:

```
cleaning_df = cleanDataFrame()
seoul_bike_sharing_demand_df = cleaning_df.run_clean()
```

Escalamiento de variables.

El escalamiento de variables, el cual es muy importante para este proyecto, se hace en la clase `compareModels` particularmente aquí:

```
class compareModels:
    def __init__(self):

        self.dataframe = cleanDataFrame().run_clean()
        self.dependent_variable = "Rented Bike Count"
        self.independent_variables = list(set(self.dataframe.columns.tolist())-{self.dependent_variable})
        self.y = np.sqrt(self.dataframe[self.dependent_variable])
        self.X = self.dataframe.drop("Rented Bike Count",axis=1)
        self.X_train, self.X_test, self.y_train, self.y_test = train_test_split(self.X, self.y, test_size = 0.2, random_state = 0)
        self.columns_table = ['Model', 'MAE', 'MSE', 'RMSE', 'R2_score', 'Adjusted_R2']
        self.comparison_table = pd.DataFrame(columns=self.columns_table)
        self.alphas = np.logspace(-4,2,50)
        self.l1_ratios = np.linspace(0.1, 1.0, 6)
        self.grid_search_comparison_table = pd.DataFrame(columns=self.columns_table)
        self.seed = 97
        self.scaler = StandardScaler() #<----- aquí
```

Es una variable de clase que después es llamada en cada método de modelación de la siguiente forma

```
X_train_scaled = self.scaler.fit_transform(self.X_train)
X_test_scaled = self.scaler.transform(self.X_test)
```

Modelado

Vamos a inicializar la clase:

```
import sys
sys.path.insert(0, '..')
from ML.modelaje import compareModels

comparacion = compareModels()
```

Regresión Logística.

Poisson (Justificación)

Nuestra variable target está compuesto de enteros no negativos:

```
comparacion.dataframe["Rented Bike Count"][0:10]
```

```
0    254
1    204
2    173
3    107
4     78
5    100
6    181
7    460
8    930
9    490
Name: Rented Bike Count, dtype: int64
```

Una regresión lineal estandar incorrectamente intentaría predecir valores negativos y además, la regresión lineal estándar asume **homoscedasticidad**. Como nuestra variable target es un conteo, esta suposición **no** se cumple, ya que la varianza crece con la media.

De hecho, podemos notar en los gráficos de abajo ese patrón de **heteroscedasticidad** con los residuales.

Ahora, un modelo GLM Poisson modela el logaritmo de la media y explícitamente asume que la varianza es igual a la media, usa un esquema de pesos ponderados. Esto en teoría debería resolver el problema de **heteroscedasticidad**. Para prestarle atención a la estacionalidad temporal, recordemos lo que hicimos con el dataframe en la etapa de limpieza:

```
import numpy as np
def make_sin_cos(self):
    self.dataframe['hour_sin'] = np.sin(2 * np.pi * self.dataframe['Hour'] / 24)
    self.dataframe['hour_cos'] = np.cos(2 * np.pi * self.dataframe['Hour'] / 24)
    self.dataframe['dow_sin'] = np.sin(2 * np.pi * self.dataframe['Day'] / 7)
    self.dataframe['dow_cos'] = np.cos(2 * np.pi * self.dataframe['Day'] / 7)

    self.dataframe['month_sin'] = np.sin(2 * np.pi * self.dataframe['Month'] / 12)
    self.dataframe['month_cos'] = np.cos(2 * np.pi * self.dataframe['Month'] / 12)
```

Esta función nos ayuda a incluir esta información de estacionalidad temporal, puesto que la hora 23 estaría “cerca” de la hora 0. Esto ayudará a que GLM Poisson tome en cuenta los “picos” de demanda de las 8:00AM y 6:00PM que notamos en el EDA. De no incluir lo de arriba, el modelo buscaría una tendencia monotonica , lo cual no brinda mucha información ni es representativo de lo que pasa en la vida real. Vamos a correr este modelo:

```
y_pred_poisson,poisson = comparacion.poisson_GLM()
comparacion.create_entry(y_pred_poisson,"GLM Poisson",comparacion.comparison_table)
comparacion.create_entry(y_pred_poisson,"GLM Poisson",comparacion.grid_search_comparison_table)
```

Ahora, corramos los demás:

Regresión Lineal

```
y_pred,linear_regressor = comparacion.linear_regression()
comparacion.plot_results(y_pred,"Linear Regression",figsize=(7,5),fontsize=4)
comparacion.create_entry(y_pred,"Linear Regression",comparacion.comparison_table)
```

Lasso

```
y_pred,lasso = comparacion.lasso()
comparacion.create_entry(y_pred,"Lasso",comparacion.comparison_table)

comparacion.plot_results(y_pred,"Lasso",figsize=(7,5),fontsize=4)
```

Ridge

```
y_pred,ridge = comparacion.ridge()
comparacion.create_entry(y_pred,"Ridge",comparacion.comparison_table)

comparacion.plot_results(y_pred,"Ridge",figsize=(7,5),fontsize=4)
```

Elastic Net

```
y_pred_elastic,elastic_net= comparacion.elastic_net()
comparacion.create_entry(y_pred_elastic,"Elastic Net",comparacion.comparison_table)
```

La tabla comparativa de estos cuatro es:

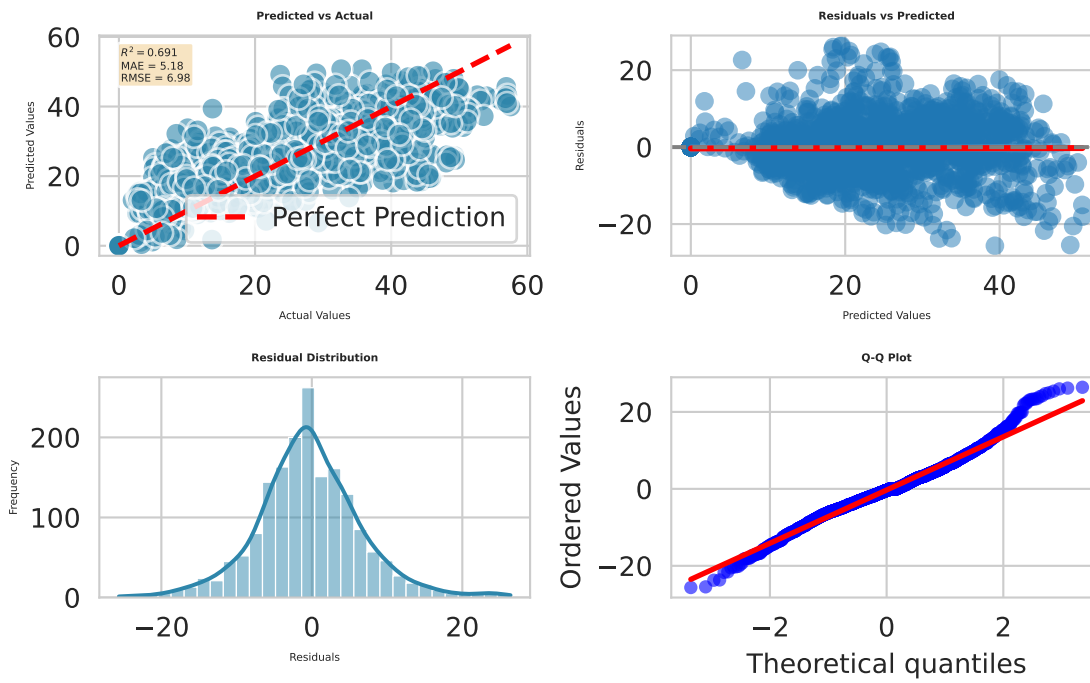
```
comparacion.comparison_table
```

	Model	MAE	MSE	RMSE	R2_score	Adjusted_R2
0	GLM Poisson	5.1805	48.7152	6.9796	0.6907	0.6871
1	Linear Regression	5.7124	54.7993	7.4027	0.6520	0.6480
2	Lasso	5.7123	54.8276	7.4046	0.6519	0.6478
3	Ridge	5.7124	54.7993	7.4027	0.6520	0.6480
4	Elastic Net	5.7124	54.8023	7.4029	0.6520	0.6480

El mejor modelo tiene, GLM Poisson , tiene este gráfico:

```
comparacion.plot_results(y_pred_poisson,"GLM Poisson",figsize=(6,4),fontsize=4)
```

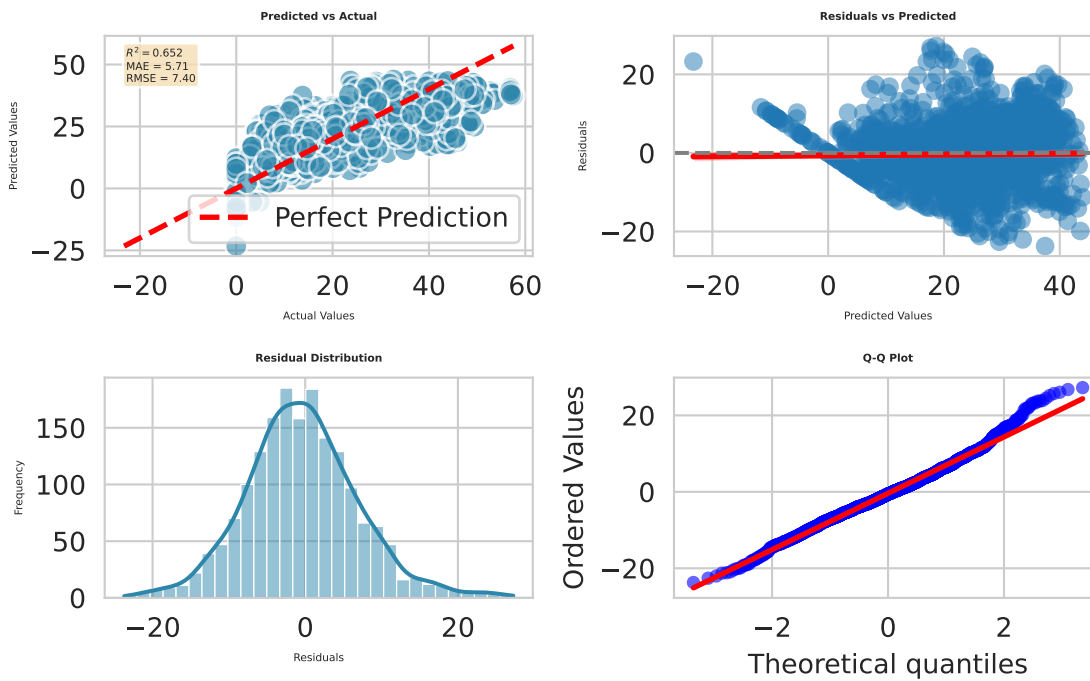
GLM Poisson



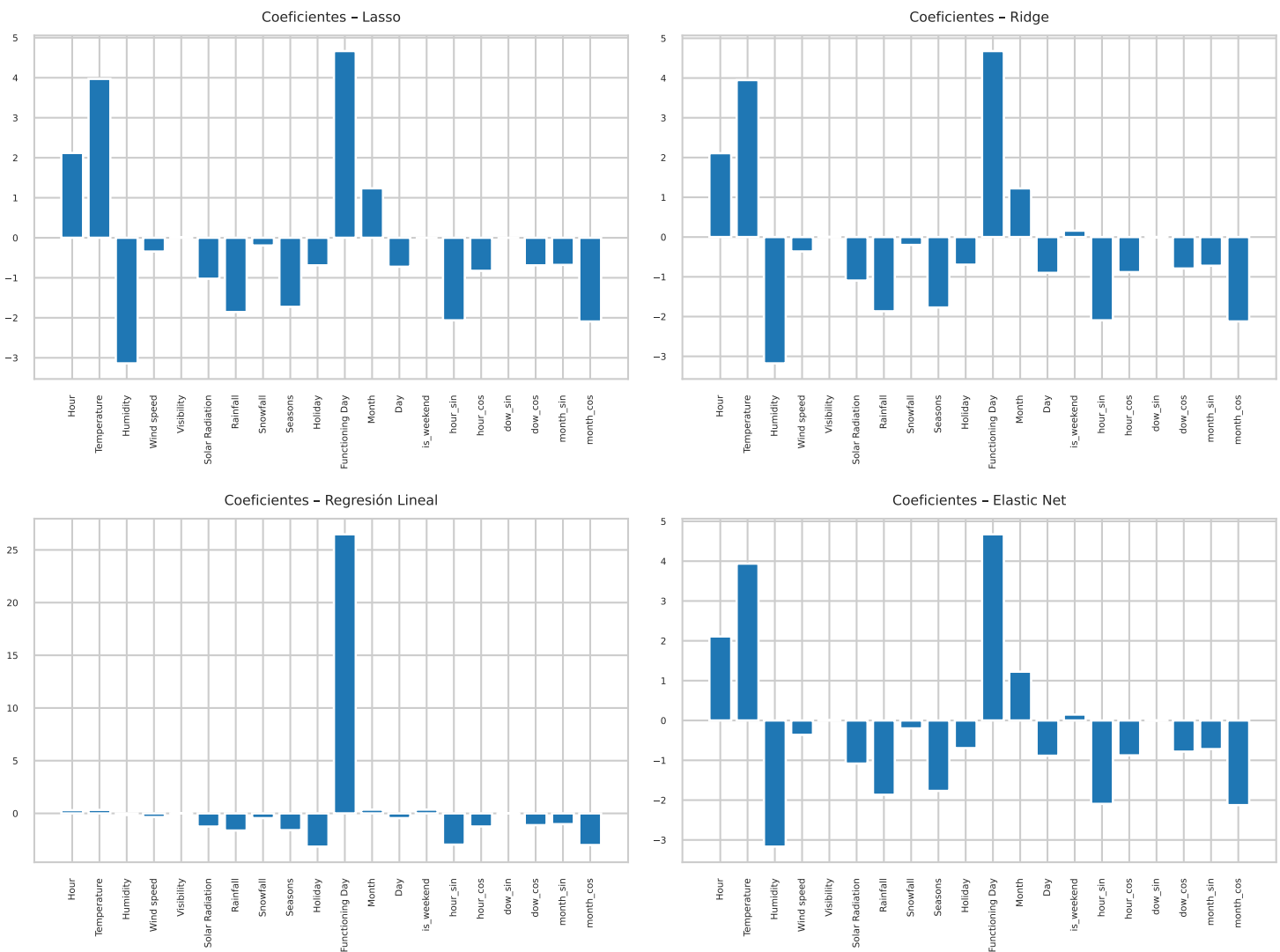
Mientras que el peor, Elastic Net, tiene este:

```
comparacion.plot_results(y_pred_elastic, "Elastic Net", figsize=(6,4), fontsize=4)
```

Elastic Net



Interpretación de coeficientes.



La interpretación es la siguiente:

- Podemos notar que **Lasso** lleva los coeficientes de las variables menos significativas a 0.
- En cambio, **Ridge** las lleva cercanas a cero pero no son cero, exactamente.
- **Elastic Net** Al ser una mezcla de ambas, puede producir coeficientes cero, en nuestro caso parece que es más similar a **Ridge**
- **Regresión Lineal**, como **Functioning Day** es una variable binaria que representa si un día el sistema de bicis estaba activo o no, que esté tan alto refleja una gran diferencia en rentas entre esos dos estados.

Modelos de Ensamble y Redes.

Empecemos con los modelos de Ensamble.

Decision Tree

```
y_pred,decision_tree = comparacion.decision_tree()
comparacion.create_entry(y_pred,"Decision Tree",comparacion.comparison_table)

comparacion.plot_results(y_pred,"Decision Tree",figsize=(6,4),fontsize=4)
```

Random Forest

```
y_pred_random_forest,random_forest = comparacion.random_forest()
comparacion.create_entry(y_pred_random_forest,"Random Forest",comparacion.comparison_table)
```


Gradient Boosting

```
y_pred,gradient_boosting = comparacion.gradient_boosting()
comparacion.create_entry(y_pred,"Gradient Boosting",comparacion.comparison_table)
comparacion.plot_results(y_pred,"Gradient Boosting",figsize=(6,4),fontsize=4)
```

Ada Boost

```
y_pred,ada_boost = comparacion.ada_boost()
comparacion.create_entry(y_pred,"AdaBoost",comparacion.comparison_table)
comparacion.plot_results(y_pred,"AdaBoost",figsize=(6,4),fontsize=4)
```

XG Boost

```
y_pred,xgboost = comparacion.xg_boost()
comparacion.create_entry(y_pred,"XGBoost",comparacion.comparison_table)
comparacion.plot_results(y_pred,"XGBoost",figsize=(6,4),fontsize=4)
```

Finalmente, con el modelo de redes (MLP)

MLP

```
y_pred,mlp = comparacion.mlp_regressor()
comparacion.create_entry(y_pred,"MLP",comparacion.comparison_table)
comparacion.plot_results(y_pred,"MLP",figsize=(6,4),fontsize=4)
```

Los resultados antes de usar validación cruzada son:

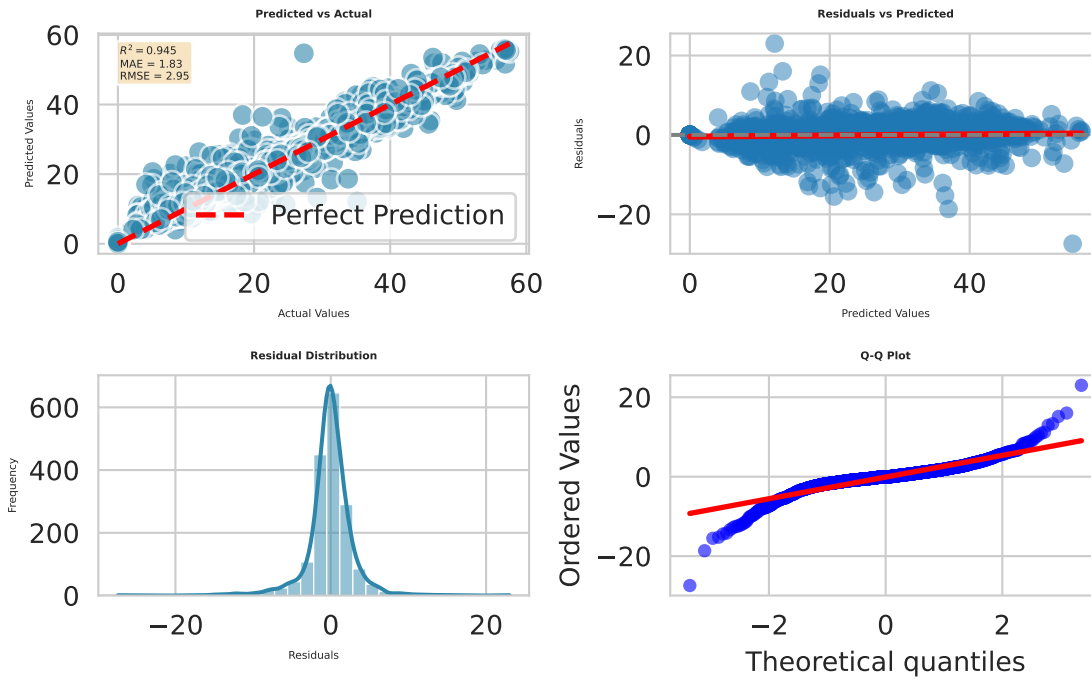
```
comparacion.comparison_table.sort_values(by='R2_score', ascending=False)
```

	Model	MAE	MSE	RMSE	R2_score	Adjusted_R2
6	Random Forest	1.8298	8.6981	2.9493	0.9448	0.9441
7	Gradient Boosting	2.0800	9.8529	3.1389	0.9374	0.9367
10	MLP	2.1855	11.2332	3.3516	0.9287	0.9278
9	XGBoost	2.9005	16.6294	4.0779	0.8944	0.8932
5	Decision Tree	3.0762	19.9571	4.4673	0.8733	0.8718
8	AdaBoost	5.7204	48.1753	6.9408	0.6941	0.6906
0	GLM Poisson	5.1805	48.7152	6.9796	0.6907	0.6871
3	Ridge	5.7124	54.7993	7.4027	0.6520	0.6480
1	Linear Regression	5.7124	54.7993	7.4027	0.6520	0.6480
4	Elastic Net	5.7124	54.8023	7.4029	0.6520	0.6480
2	Lasso	5.7123	54.8276	7.4046	0.6519	0.6478

y el grafico del mejor modelo, **RandomForest**, es :

```
comparacion.plot_results(y_pred_random_forest,"Random Forest",figsize=(6,4),fontsize=4)
```

Random Forest



Uso de Validación Cruzada

La siguiente función `re_run_with_better_params()` extraerá los parámetros y correrá los modelos con dichos parámetros. La forma en la cual lo hace es la siguiente, consigue todos los parámetros con la siguiente función

```
def grid_searching(self,scoring="r2"):
    #Escalamiento variables.
    X_train_scaled = self.scaler.fit_transform(self.X_train)

    #lasso
    lasso_param_grid = {"alpha":self.alphas}
    lasso = Lasso(max_iter=10000)
    lasso_grid = GridSearchCV(lasso,lasso_param_grid,cv=5,scoring="r2")
    lasso_grid.fit(X_train_scaled, self.y_train)

    #ridge
    ridge_param_grid = {"alpha":self.alphas}
    ridge = Ridge(max_iter=10000)
    ridge_grid = GridSearchCV(ridge,ridge_param_grid,cv=5,scoring="r2")
    ridge_grid.fit(X_train_scaled, self.y_train)

    #elastic net
    elastic_net_param_grid = {
        "alpha": self.alphas,
        "l1_ratio": self.l1_ratios
    }
    elastic_net = ElasticNet(max_iter=10000)
    elastic_net_grid = GridSearchCV(elastic_net,elastic_net_param_grid,cv=5,scoring="r2")
    elastic_net_grid.fit(X_train_scaled, self.y_train)

    #decision tree
    decision_tree_param_grid = {
        'max_depth': [3, 5, 7, 9, 11, 15, None],
        'min_samples_split': [2, 5, 10, 20],
        'min_samples_leaf': [1, 2, 5, 10]
    }
    decision_tree = DecisionTreeRegressor(random_state=self.seed)
    decision_tree_grid = GridSearchCV(decision_tree,decision_tree_param_grid,cv=5,scoring="r2")
    decision_tree_grid.fit(self.X_train,self.y_train)

    #random forest
    random_forest_param_grid = {
        'n_estimators': [100, 200],
        'max_depth': [5, 10, 15, None],
        'min_samples_split': [2, 5, 10]
    }
    random_forest = RandomForestRegressor(random_state=self.seed)
    random_forest_grid = GridSearchCV(random_forest,random_forest_param_grid,cv=5,scoring="r2")
```

```

random_forest_grid.fit(self.X_train,self.y_train)
#gradient_boosting
gradient_boosting_param_grid = {
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'n_estimators': [100, 200, 300],
    'max_depth': [3, 5, 7]
}
gradient_boosting = GradientBoostingRegressor(random_state=self.seed)
gradient_boosting_grid = GridSearchCV(gradient_boosting,gradient_boosting_param_grid,cv=5,scoring="r2")
gradient_boosting_grid.fit(self.X_train,self.y_train)

#AdaBoost
ada_param_grid = {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.5, 1.0, 2.0],
    'loss': ['linear', 'square', 'exponential']
}
ada = AdaBoostRegressor(random_state=self.seed)
ada_grid = GridSearchCV(ada, ada_param_grid, cv=5, scoring='r2')
ada_grid.fit(X_train_scaled, self.y_train)

#XGBoost
xgb_param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.1, 0.2],
    'subsample': [0.7, 0.8, 1.0]
}
xgb = XGBRegressor(random_state=self.seed, verbosity=0)
xgb_grid = GridSearchCV(xgb, xgb_param_grid, cv=5, scoring='r2')
xgb_grid.fit(X_train_scaled, self.y_train)

#MLP
mlp_param_grid = {
    'hidden_layer_sizes': [(50,), (100,), (50, 25)],
    'activation': ['relu', 'tanh'],
    'alpha': [0.0001, 0.001, 0.01],
    'learning_rate': ['constant', 'adaptive']
}
mlp = MLPRegressor(max_iter=1000, random_state=self.seed, early_stopping=True)
mlp_grid = GridSearchCV(mlp, mlp_param_grid, cv=5, scoring='r2')
mlp_grid.fit(X_train_scaled, self.y_train)

best_params = {
    "lasso":lasso_grid,
    "ridge": ridge_grid,
    "elastic_net": elastic_net_grid,
    "decision_tree":decision_tree_grid,
    "random_forest": random_forest_grid,
    "gradient_boost": gradient_boosting_grid,
    "ada_boost": ada_grid,
    "xg_boost": xgb_grid,
    "mlp": mlp_grid
}

print(best_params)

return best_params

```

Y después vuelve a correr los modelos con los mejores parámetros. Vamos a ejecutarlo, toma en promedio **20** minutos así que usaremos un diccionario con los resultados de una previa ejecución:

```

import json
with open('best_params.json', 'r') as f:
    best_params_dict = json.load(f)

```

```

y_predictions,best_params = comparacion.re_run_with_better_params(best_params_dict)

```

A continuación, una comparación entre ambas tablas, la primera es con **GridSearchCV**:

	Model	MAE	MSE	RMSE	R2_score	Adjusted_R2
8	XGBoost (Grid Search)	1.6099	6.9595	2.6381	0.9558	0.9553
6	Gradient Boost (Grid Search)	1.6616	7.5023	2.7390	0.9524	0.9518
5	Random Forest (Grid Search)	1.8320	8.7288	2.9544	0.9446	0.9439

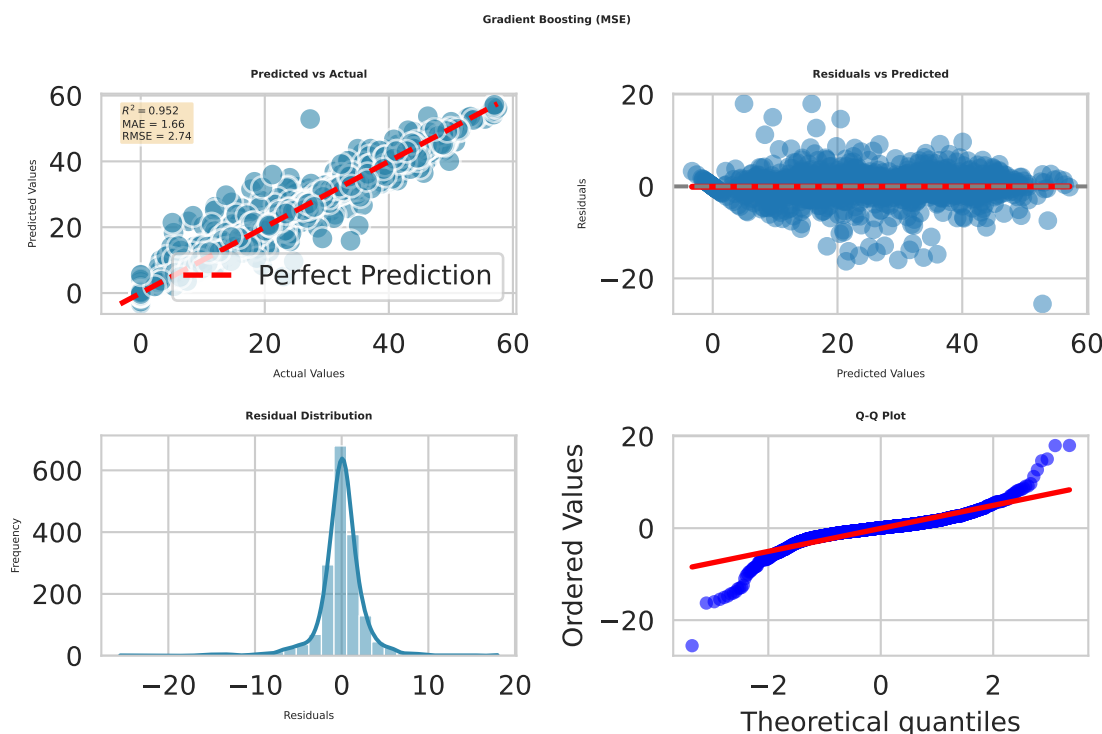
	Model	MAE	MSE	RMSE	R2_score	Adjusted_R2
9	MLP (Grid Search)	1.8130	8.7986	2.9662	0.9441	0.9435
4	Decision Tree (Grid Search)	2.5514	15.6495	3.9559	0.9006	0.8995
7	AdaBoost (Grid Search)	5.1827	42.7833	6.5409	0.7283	0.7252
0	GLM Poisson	5.1805	48.7152	6.9796	0.6907	0.6871
2	Ridge (Grid Search)	5.7126	54.8128	7.4036	0.6520	0.6479
1	Lasso (Grid Search)	5.7120	54.8436	7.4056	0.6518	0.6477
3	Elastic Net (Grid Search)	5.7120	54.8436	7.4056	0.6518	0.6477

	Model	MAE	MSE	RMSE	R2_score	Adjusted_R2
6	Random Forest	1.8298	8.6981	2.9493	0.9448	0.9441
7	Gradient Boosting	2.0800	9.8529	3.1389	0.9374	0.9367
10	MLP	2.1855	11.2332	3.3516	0.9287	0.9278
9	XGBoost	2.9005	16.6294	4.0779	0.8944	0.8932
5	Decision Tree	3.0762	19.9571	4.4673	0.8733	0.8718
8	AdaBoost	5.7204	48.1753	6.9408	0.6941	0.6906
0	GLM Poisson	5.1805	48.7152	6.9796	0.6907	0.6871
3	Ridge	5.7124	54.7993	7.4027	0.6520	0.6480
1	Linear Regression	5.7124	54.7993	7.4027	0.6520	0.6480
4	Elastic Net	5.7124	54.8023	7.4029	0.6520	0.6480
2	Lasso	5.7123	54.8276	7.4046	0.6519	0.6478

Funciones de perdida.

De los mejores modelos, vamos a comparar MAE vs MSE, empecemos con GradientBoost ya que admite un parametro loss para especificar MAE o MSE.

```
import pandas as pd
tabla_comparativa_gradientboost = pd.DataFrame(columns=comparacion.columns_table)
y_pred,gradient_boosting = comparacion.gradient_boosting(
    best_params["gradient_boost"]["learning_rate"],
    best_params["gradient_boost"]["max_depth"],
    best_params["gradient_boost"]["n_estimators"],
    loss="squared_error")
tabla_comparativa_gradientboost=comparacion.create_entry(y_pred,"Gradient Boosting (MSE)",tabla_comparativa_gradientboost)
comparacion.plot_results(y_pred,"Gradient Boosting (MSE)",figsize=(6,4),fontsize=4)
tabla_comparativa_gradientboost
```

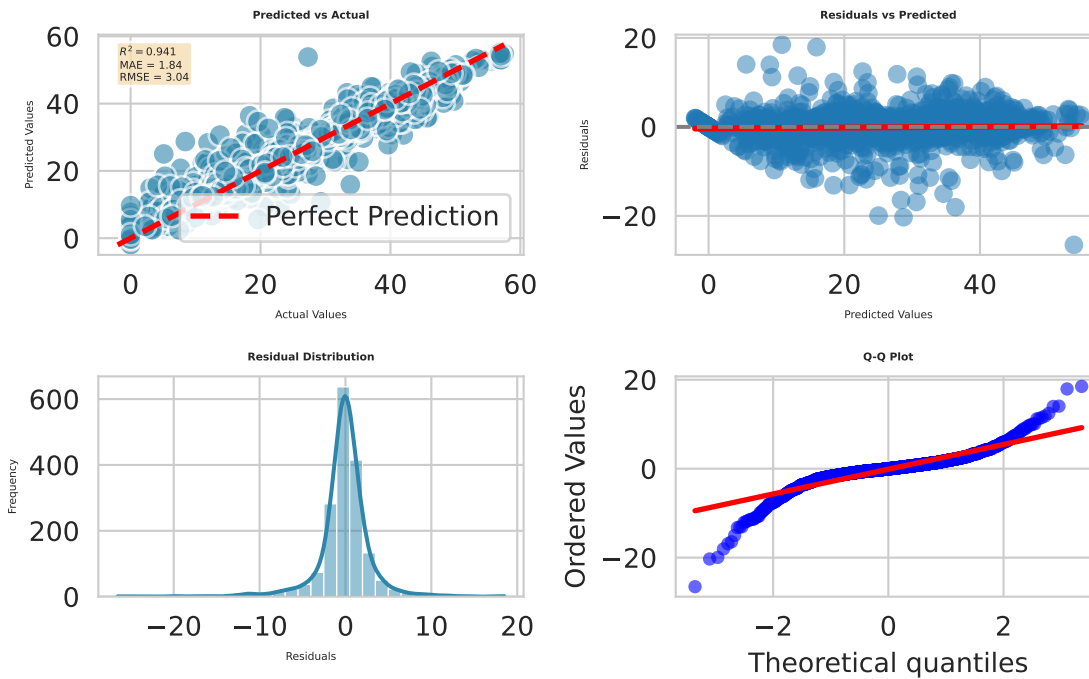


	Model	MAE	MSE	RMSE	R2_score	Adjusted_R2
0	Gradient Boosting (MSE)	1.6616	7.5023	2.739	0.9524	0.9518

```
import pandas as pd
y_pred,gradient_boosting = comparacion.gradient_boosting(
    best_params["gradient_boost"]["learning_rate"],
    best_params["gradient_boost"]["max_depth"],
    best_params["gradient_boost"]["n_estimators"],
    loss="absolute_error")

tabla_comparativa_gradientboost=comparacion.create_entry(y_pred,"Gradient Boosting (MAE)",tabla_comparativa_gradientboost)
comparacion.plot_results(y_pred,"Gradient Boosting (MAE)",figsize=(6,4),fontsize=4)
```

Gradient Boosting (MAE)



tabla_comparativa_gradientboost

	Model	MAE	MSE	RMSE	R2_score	Adjusted_R2
0	Gradient Boosting (MSE)	1.6616	7.5023	2.7390	0.9524	0.9518
1	Gradient Boosting (MAE)	1.8373	9.2638	3.0437	0.9412	0.9405

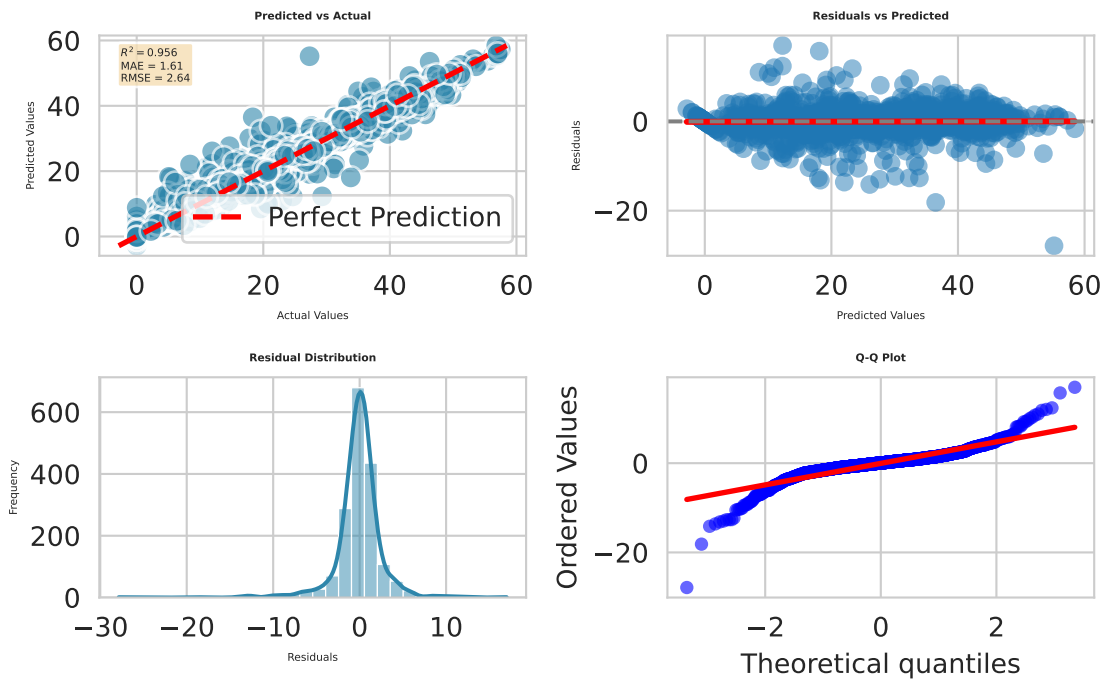
Tenemos que para **GradientBoost** MSE es mejor.

Ahora intentemos con el otro modelo que resultó ser muy bueno, **XGBoost**:

```
tabla_comparativa_xgboost = pd.DataFrame(columns=comparacion.columns_table)
y_pred_xgboost,xgboost = comparacion.xg_boost(
    best_params["xg_boost"]["learning_rate"],
    best_params["xg_boost"]["max_depth"],
    best_params["xg_boost"]["n_estimators"],
    objective="reg:squarederror")

tabla_comparativa_xgboost=comparacion.create_entry(y_pred_xgboost,"XGBoost (MSE)",tabla_comparativa_xgboost)
comparacion.plot_results(y_pred_xgboost,"XGBoost (MSE)",figsize=(6,4),fontsize=4)
tabla_comparativa_xgboost
```

XGBoost(MSE)



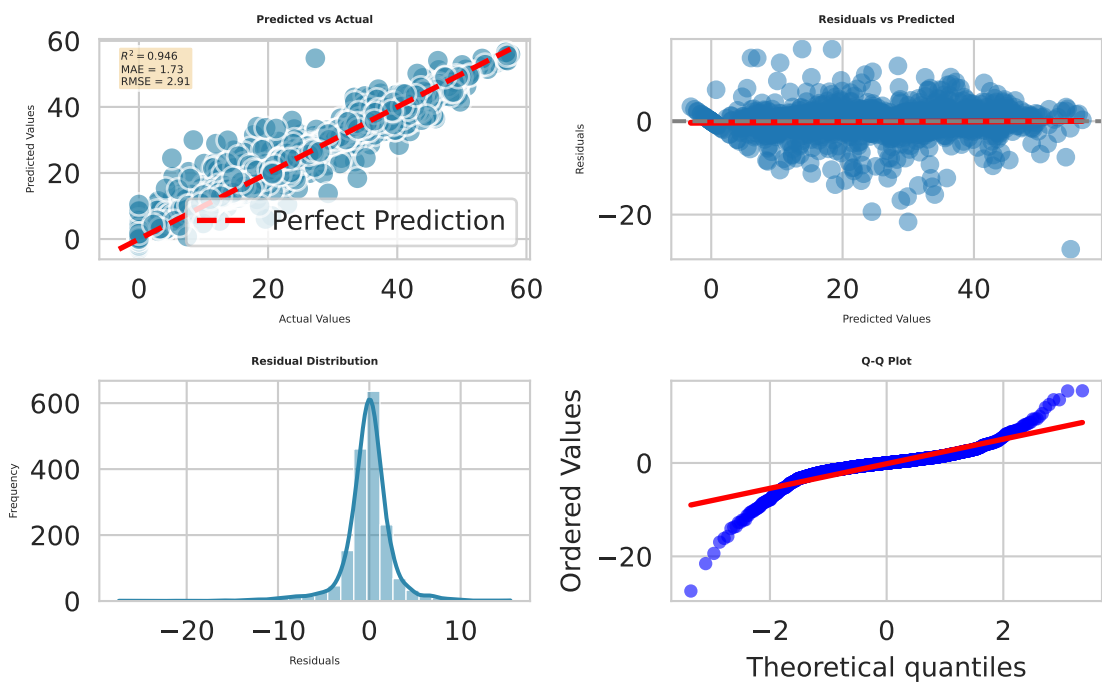
	Model	MAE	MSE	RMSE	R2_score	Adjusted_R2
0	XGBoost (MSE)	1.6099	6.9595	2.6381	0.9558	0.9553

```

y_pred,xgboost = comparacion.xg_boost(
    best_params["xg_boost"]["learning_rate"],
    best_params["xg_boost"]["max_depth"],
    best_params["xg_boost"]["n_estimators"],
    objective="reg:absoluteerror")
tabla_comparativa_xgboost=comparacion.create_entry(y_pred,"XGBoost (MAE)",tabla_comparativa_xgboost)
comparacion.plot_results(y_pred,"XGBoost (MAE)",figsize=(6,4),fontsize=4)
tabla_comparativa_xgboost

```

XGBoost(MAE)



	Model	MAE	MSE	RMSE	R2_score	Adjusted_R2
0	XGBoost (MSE)	1.6099	6.9595	2.6381	0.9558	0.9553
1	XGBoost (MAE)	1.7311	8.4488	2.9067	0.9464	0.9457

En ambos casos, correrlos con MSE resultó ser mejor.

La razón por la cual MSE resultó ser mejor es por como procesamos previamente la variable dependiente (**Rented Bike Count**) recordemos que hice lo siguiente:

```
self.y = np.sqrt(self.dataframe[self.dependent_variable])
```

De esta manera redujimos la influencia de los picos extremos de demanda, los cuales vimos en la sección de EDA que ocurrían alrededor de los horarios de entrada y salida de oficinas. Después de la raíz cuadrada los valores se comprimen, por ejemplo si cerca las 8:00AM se rentan más de 2000 bicis pero a las 12:00AM se rentan 500, la raíz cuadrada los manda respectivamente a 44.7 y a 22.4 por lo que la distribución se vuelve más simétrica y normal.

Lo quisimos hacer de esta manera para suavizar dichos picos de demanda y no considerarlos outliers puesto que dan mucha información acerca de patrones de uso.

Interpretación de resultados y conclusión

Al final la tabla quedó así:

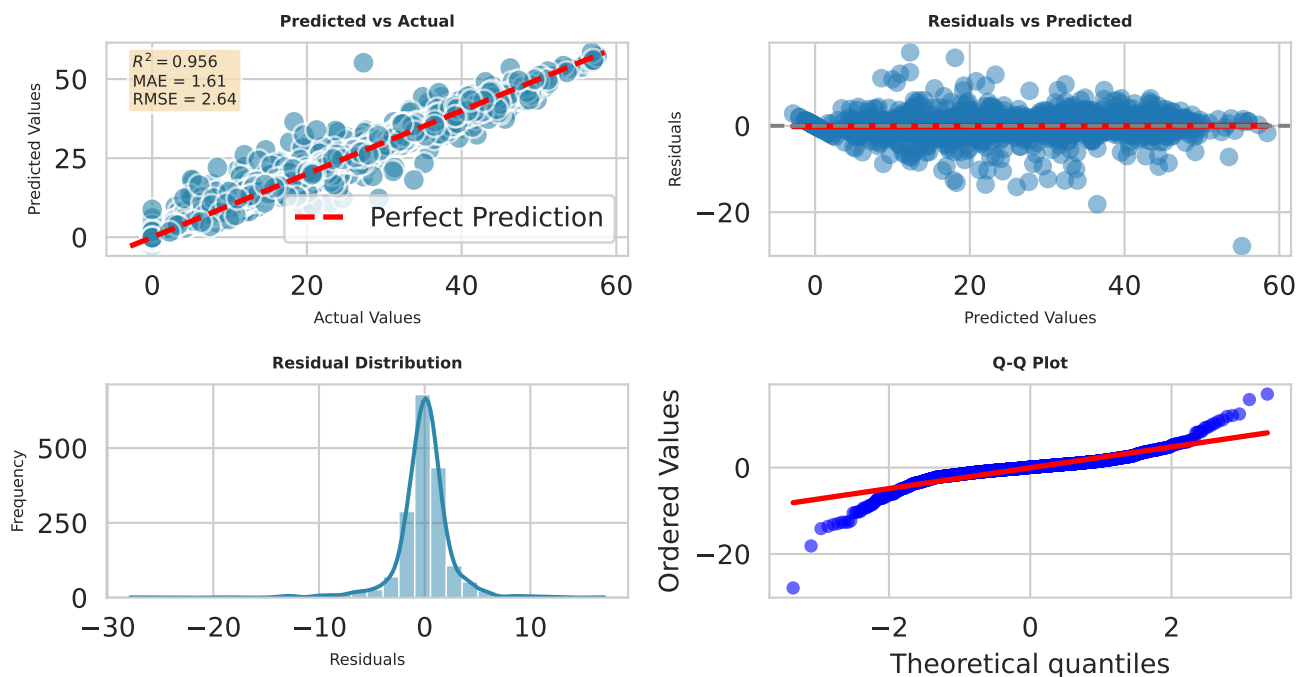
```
comparacion.grid_search_comparison_table.sort_values(by='R2_score', ascending=False)
```

	Model	MAE	MSE	RMSE	R2_score	Adjusted_R2
8	XGBoost (Grid Search)	1.6099	6.9595	2.6381	0.9558	0.9553
6	Gradient Boost (Grid Search)	1.6616	7.5023	2.7390	0.9524	0.9518
5	Random Forest (Grid Search)	1.8320	8.7288	2.9544	0.9446	0.9439
9	MLP (Grid Search)	1.8130	8.7986	2.9662	0.9441	0.9435
4	Decision Tree (Grid Search)	2.5514	15.6495	3.9559	0.9006	0.8995
7	AdaBoost (Grid Search)	5.1827	42.7833	6.5409	0.7283	0.7252
0	GLM Poisson	5.1805	48.7152	6.9796	0.6907	0.6871
2	Ridge (Grid Search)	5.7126	54.8128	7.4036	0.6520	0.6479
1	Lasso (Grid Search)	5.7120	54.8436	7.4056	0.6518	0.6477
3	Elastic Net (Grid Search)	5.7120	54.8436	7.4056	0.6518	0.6477

Con la grafica del mejor modelo así:

```
comparacion.plot_results(y_pred_xgboost,"XGBoost",figsize=(7,4),fontsize=6)
```

XGBoost



Conclusión

Existe una diferencia muy marcada entre las dos familias de modelos evaluados:

- Modelos Lineales Regularizados (Lasso, Ridge, Elastic Net) + Poisson
- Modelos de Ensamble basados en árboles + MLP

Poisson ofreció una mejora significativa con respecto de Lasso, Ridge y Elastic Net, probablemente porque dado los supuestos es el modelo adecuado si nos limitáramos a Modelos Lineales Regularizados (es una variable de conteo, etc) pero aún así está muy lejos del segundo grupo mencionado en la lista de arriba en términos de *performance*.

La demanda de bicicletas tiene que ser no-lineal debido a la relación entre distintas variables como lo puede ser la humedad, la visibilidad, etc, por lo que los modelos del segundo grupo mencionado son mejores, estos captan las complejas interacciones y la fuerte estacionalidad horaria.

Además, usar MSE y hacer la modificación de raíz cuadrada a nuestra variable dependiente resultó ser un acierto:

```
self.y = np.sqrt(self.dataframe[self.dependent_variable])
```

Dado que logramos suavizar los picos de demanda horaria sin removerlos o considerarlos como *outliers*. También, incluir columnas de estacionalidad temporal como lo son los senos y cosenos usando las horas y días fue un acierto también

```
def make_sin_cos(self):
    self.dataframe['hour_sin'] = np.sin(2 * np.pi * self.dataframe['Hour'] / 24)
    self.dataframe['hour_cos'] = np.cos(2 * np.pi * self.dataframe['Hour'] / 24)
    self.dataframe['dow_sin'] = np.sin(2 * np.pi * self.dataframe['Day'] / 7)
    self.dataframe['dow_cos'] = np.cos(2 * np.pi * self.dataframe['Day'] / 7)

    self.dataframe['month_sin'] = np.sin(2 * np.pi * self.dataframe['Month'] / 12)
    self.dataframe['month_cos'] = np.cos(2 * np.pi * self.dataframe['Month'] / 12)
```

Dado que con esto prestamos atención a la estacionalidad temporal, contextualizamos así las variables de hora, día y mes.